

```
#include <iostream>
#include <sstream>
#include <string>
#include <vector>
#include <cmath>
#include <cctype>
#include <cstdlib>
using namespace std;
```

```
string lowercase(string s)
{
    for (int n = 0; n < s.length(); n+=1)
    {
        s[n] = tolower(s[n]);
    }

    return s;
}
```

```
bool only_letters(const string& s)
{
    for (int n = 0; n < s.length(); n+=1)
    {
        if (!isalpha(s[n]))
        {
            return false;
        }
    }

    return true;
}
```

```
struct Memory
{
    vector<string> vnames;
    vector<double> vvalues;

    int variable(const string& name) const
    {
        for (int n = 0; n < vnames.size(); n+=1)
        {
            if (vnames[n] == name)
```

```

        return n;
    }

    return -1;
}

bool get(const string& name, double& value) const
{
    int index = variable(name);
    if (index == -1)
        return false;
    value = vvalues[index];
    return true;
}

void set(const string& name, double value)
{
    int index = variable(name);

    if (index == -1)
    {
        vnames.push_back(name);
        vvalues.push_back(value);
    }
    else
    {
        vvalues[index] = value;
    }
}

};

int main()
{
    vector<double> stack;
    Memory mem;

    mem.set("pi", acos(-1.0));

    string line;
    cout << "Hello, Welcome to Your Reverse Polish Calculator (click Ctrl-D to quit) :\n";

```

```

while (true)
{
    cout << "> ";
    if (!getline(cin, line))
    {
        break; // ctrl-D
    }

    stack.clear();
    istringstream input(line);
    string token;
    bool error = false;

    while (input >> token)
    {
        char* endptr;
        double number = strtod(token.c_str(), &endptr);

        if (*endptr == '\0')
        {
            stack.push_back(number);
            continue;
        }

        if (token == "->")
        {
            if (stack.empty())
            {
                cout << "uh oh, error - nothing to assign\n";
                error = true;
                break;
            }
            string name;
            if (!(input >> name))
            {
                cout << "uh oh, error - missing variable name\n";
                error = true;
                break;
            }
            string name;
            if (!(input >> name))
            {
                cout << "uh oh, error - missing variable name\n";
                error = true;
            }
        }
    }
}

```

```

        break;
    }
    name = lowercase(name);
    if (!only_letters(name))
    {
        cout << "uh oh, error - invalid variable name\n";
        error = true;
        break;
    }
    mem.set(name, stack.back());
    continue;
}

if (token == "+" || token == "-" || token == "*" || token == "/" || token == "^")
{
    if (stack.size() < 2)
    {
        cout << "uh oh, error - not enough operands\n";
        error = true;
        break;
    }

    double b = stack.back(); stack.pop_back();
    double a = stack.back(); stack.pop_back();

    if (token == "/" && b == 0)
    {
        cout << "uh oh, error - division by zero\n";
        error = true;
        break;
    }

    if (token == "+") stack.push_back(a + b);
    else if (token == "-") stack.push_back(a - b);
    else if (token == "*") stack.push_back(a * b);
    else if (token == "/") stack.push_back(a / b);
    else if (token == "^") stack.push_back(pow(a, b));
    continue;
}

string t = lowercase(token);
double val;

```

```

if (mem.get(t, val))
{
    stack.push_back(val);
    continue;
}

if (stack.empty())
{
    cout << "uh oh, error - stack empty\n";
    error = true;
    break;
}

double x = stack.back();
stack.pop_back();

if (t == "sqrt") stack.push_back(sqrt(x));
else if (t == "sin") stack.push_back(sin(x));
else if (t == "cos") stack.push_back(cos(x));
else if (t == "tan") stack.push_back(tan(x));
else if (t == "asin") stack.push_back(asin(x));
else if (t == "acos") stack.push_back(acos(x));
else if (t == "atan") stack.push_back(atan(x));
else if (t == "sinh") stack.push_back(sinh(x));
else if (t == "cosh") stack.push_back(cosh(x));
else if (t == "tanh") stack.push_back(tanh(x));
else if (t == "log") stack.push_back(log(x));
else if (t == "log10") stack.push_back(log10(x));
else if (t == "exp") stack.push_back(exp(x));
else if (t == "abs") stack.push_back(fabs(x));
else if (t == "floor") stack.push_back(floor(x));
else if (t == "ceil") stack.push_back(ceil(x));
else if (t == "round") stack.push_back(round(x));
else if (t == "cbrt") stack.push_back(cbrt(x));
else if (t == "sign") stack.push_back((x>0)-(x<0));
else
{
    cout << "uh oh, error - unknown variable\n";
    error = true;
    break;
}
}

if (!error)

```

```

    {
        if (stack.size() == 1) cout << stack.back() << endl;
        else cout << "uh oh, error - invalid expression\n";
    }
}
}

```

```

taa182@rabbit:~/ECE218/RPC % CC RPC.cpp
taa182@rabbit:~/ECE218/RPC % a.out
Hello, Welcome to Your Reverse Polish Calculator (click Ctrl-D to quit) :)
> 3 4 5 + *
27
> 3 4 5 + * -> radius
27
> radius 2 ^ pi *
2290.22
> 1 2 3 4 5 * * * * -> x 2 *
240
> pi 0.25 * sin
0.707107
> 2 4 5 1 - - / 3 +
uh oh, error - division by zero
> 6.254175 3.745824 +
10
> 16 sqrt
4
> 20 4 /
5
> 25 sqrt sin 1 +
0.0410757
> 2 3 + 4 1 - * pi /
4.77465
> pi 4 / tan 16 sqrt +
5
> 2 3 ^

```

8

> 10 10 * log 2 +

6.60517

> 0.5 acos sin

0.866025

> 7 10 - abs 2 *

6

> pi pi * 1 + sqrt sin

-0.154692

>